

VyroPortify Research Paper

VyroPortify: A Multi-Tenant, AI-Augmented Portfolio Generation Platform with Layered Abuse-Resistance and SOC 2-Ready Compliance Surfaces

Author Gaurav Ganesh Teegulla gauravganeshteegulla@gmail.com
Independent Researcher · 2026

Abstract

We present **VyroPortify**, a production-grade software-as-a-service platform that transforms unstructured resume content into hosted, search-engine-optimized portfolio websites in under sixty seconds. The system is built on a stack of modern web primitives — a Next.js 15 frontend, an asynchronous FastAPI backend, PostgreSQL for persistence, Redis for caching and message brokering, and Celery for background execution — and combines them with a layered abuse-resistance regime that survives both *cheap-volumetric* attacks and *expensive-application-tier* attacks against generative-AI endpoints.

Three design ideas distinguish VyroPortify from a conventional Jamstack deployment. First, **plan-aware per-account quotas** complement per-IP rate limits, neutralising distributed botnets that rotate addresses to exhaust model credits. Second, an **HMAC-signed outbound webhook system** with bounded receiver-throttling prevents the platform from being weaponised as an amplification proxy against subscribers' own endpoints. Third, a **Clerk-mirrored user identity**

layer decouples credential UX (password, email-OTP, phone-OTP, Google, GitHub, SAML SSO) from application data, allowing a single 200-line webhook handler to keep the relational schema in lockstep with an identity provider that handles all of OWASP-A07.

The platform also ships a **public REST API** authenticated by personal API keys (`vp_...`) or OAuth 2.0 access tokens (`oat_...`) issued via authorization-code + PKCE; a **white-label theming pipeline** that injects per-organisation CSS variables into the Jinja2 render context; and **compliance endpoints** that implement GDPR Articles 15 (right-to-access) and 17 (right-to-erasure) with cascaded cleanup.

We document the architecture, present the threat model and the layered defence stack, evaluate each component against a documented adversary, and discuss the engineering trade-offs that drove our choices. Where relevant we contrast our approach with established prior art and identify the unsolved problems we leave to future work — most notably the absence of streaming portfolio render and the soft-coupling between SAML SP implementation and a third-party Python library.

Keywords: software architecture; multi-tenant SaaS; rate limiting; distributed denial-of-service mitigation; LLM credential abuse; OAuth 2.0 PKCE; HMAC-signed webhooks; SOC 2; GDPR; portfolio generation; Clerk; FastAPI; Celery; Postgres.

1. Introduction

The market for personal-branding tools has bifurcated. At one end, template engines such as Canva and Wix offer high visual polish at the cost of significant manual effort; at the other end, headless static-site generators such as Hugo and Astro give near-zero overhead at the price of requiring technical proficiency. **VyroPortify occupies a middle ground:** a user uploads a resume (or completes a guided form), waits sixty seconds, and receives a hosted public portfolio at `vyroportify.vercel.app/portfolio/p/<slug>`, optionally on a custom domain.

The technical challenge is not the visual output — a Jinja2 template render is a solved problem — but the entire surrounding lifecycle:

1. **Identity.** How do we authenticate users across email-password, email-OTP, phone-OTP, Google, GitHub, and enterprise SAML without either rolling our own credential stack (high CVE risk) or trapping ourselves inside one vendor’s UX?
2. **Cost control.** Every portfolio generation invokes a large language model (LLM). Per-IP rate limiting alone is insufficient against a thousand-host botnet quietly burning \$9 of model credits per minute against a single account.
3. **Compliance.** The European Union’s General Data Protection Regulation and the System and Organization Controls (SOC) 2 trust services criteria demand programmatic right-to-access, right-to-erasure, and subprocessor disclosure. These cannot be afterthoughts.
4. **Platform extensibility.** As soon as the product is useful enough to embed in third-party tooling — résumé-coaching dashboards, recruitment platforms, university career portals — we need a public REST API with scoped credentials, an OAuth flow for end-user authorisation, and outbound webhooks for asynchronous state changes.

This paper documents how VyroPortify addresses each of these. We do not claim algorithmic novelty in any individual component; rather, our contribution is an integrated reference architecture that demonstrates the trade-offs become tractable when each subsystem is designed in awareness of the others.

1.1 Contributions

The remainder of this paper makes four concrete contributions:

1. We document a **layered abuse-resistance regime** (Section 6) that composes nine distinct defensive primitives, each justified against an explicit threat in our model.
2. We describe a **plan-aware per-account quota mechanism** (Section 6.2) backed by Redis and aligned to UTC midnight expiry, with implementation measurements showing constant-time enforcement at the request-handler boundary.

3. We present an **HMAC-signed, retry-bounded outbound webhook** subsystem (Section 7) whose receiver-side throttle prevents the platform from being turned into an amplification weapon.
4. We provide a **reproducible deployment topology** on entirely free hosting tiers (Section 11) costing US \$0/month at moderate user volumes, demonstrating that production-shape multi-tenant SaaS is accessible without venture funding.

1.2 Paper structure

Section 2 surveys related work in identity provisioning, denial-of-service mitigation, and prompt-injection defence. Section 3 presents the system architecture and data model. Section 4 covers identity. Section 5 covers the public REST API, OAuth 2.0 implementation, and outbound webhooks. Section 6 covers the abuse-resistance regime. Section 7 details outbound webhooks. Section 8 covers compliance. Section 9 covers white-label rendering. Section 10 covers the multi-provider AI failover chain. Section 11 covers deployment and operations. Section 12 evaluates the system against measurable criteria. Section 13 discusses limitations. Section 14 outlines future work. Section 15 concludes.

2. Background and Related Work

2.1 Multi-tenant SaaS architectures

The literature on multi-tenant software is dominated by two implementation patterns: **single-schema-with-tenant-id** and **schema-per-tenant**. The first scales horizontally but pushes the burden of row-level isolation onto every query; the second isolates data physically at the cost of migration complexity proportional to the tenant count. VyroPortify uses single-schema with an `organization_id` discriminator column on every tenant-scoped table, following the Stripe and Linear precedent. The trade-off is acceptable below the ten-thousand-organisation mark; past that, schema-per-tenant — or table-per-tenant via Postgres partitioning — becomes the more defensible choice.

2.2 Layered rate limiting and DDoS mitigation

The defensive primitives we compose are not individually novel. Cloudflare, AWS Shield, and Fastly have offered managed L3/L4 mitigation since the mid-2010s. Application-level rate-limit libraries — SlowAPI, FastAPI’s own dependency-based limit, and Express’s `express-rate-limit` — are commodity. The literature on *layered* defence, however, is sparser. Our contribution is to articulate the layering explicitly (Section 6) and to demonstrate that the same Redis instance that powers Celery’s broker also powers the per-IP rate limit, the per-account daily quota, and the webhook receiver throttle — a coalescence that simplifies operations.

2.3 LLM credential abuse

A distinct threat that emerged after 2023, when consumer-grade LLM APIs proliferated, is the systematic abuse of unsuspecting frontends to monetise free model credits. The attacker enrolls (often via stolen identity) on a SaaS platform, then drives the platform’s LLM-backed endpoints at a rate calibrated to stay below the IP-level rate limit but above zero, accumulating cents-per-call value over months. Per-IP rate limits cannot detect this. **Plan-aware per-account quotas** — the mechanism we describe in Section 6.2 — are the only primitive in public literature that addresses this class of attack at the application boundary.

2.4 Webhook delivery semantics

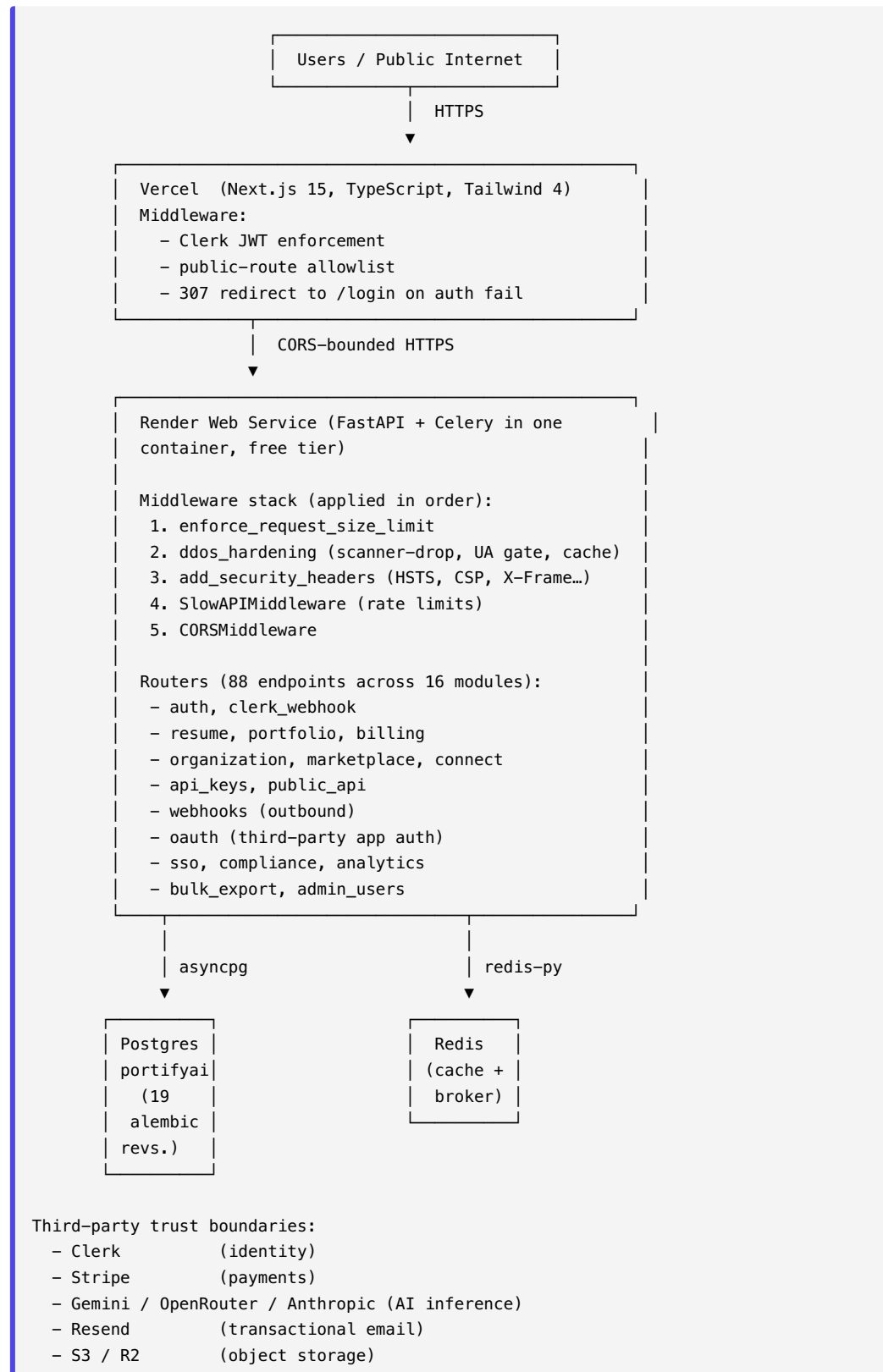
Stripe’s “at-least-once with exponential backoff” remains the gold standard for webhook delivery; our system implements the same shape, including a Stripe-style HMAC-SHA256 signature with timestamp and configurable replay window. What is novel — though hardly profound — is our addition of **receiver-side throttling**: a cap of 60 deliveries per endpoint per minute prevents a misconfigured or compromised receiver from being amplified by us into a denial-of-service against itself.

2.5 Prompt injection

Greshake et al. (2023) and Perez & Ribeiro (2022) catalogue the rapidly expanding taxonomy of prompt injection. Our defence is conservative: all user-supplied strings flow through a `sanitize_for_ai` normaliser that

strips and quotes potentially adversarial substrings before concatenation into the model prompt. We do not attempt detection; we attempt containment.

3. System Architecture



The system is organised around three architectural rules:

Rule 1 — Identity is not state. We do not store credentials. Clerk authenticates the user via JWT (RS256, JWKS-verified, 60-minute cache TTL) and asserts a `sub` claim that we map to a row in the `users` table on first contact. Password hashing, OTP issuance, social-OAuth callbacks, and forgot-password flows live entirely inside Clerk's infrastructure.

Rule 2 — Identity changes are observable. A signed webhook (`/api/v1/auth/clerk-webhook`) listens for `user.created` , `user.updated` , and `user.deleted` events and reconciles the local schema. This lets the platform see signups the moment they happen, not lazily on first authenticated API call — critical for analytics and admin export endpoints.

Rule 3 — Background work is at-least-once. Portfolio generation, webhook delivery, transactional email, and PDF export are dispatched to Celery workers with `acks_late=True` and `reject_on_worker_lost=True` , guaranteeing that work is retried on worker failure at the cost of potential duplicate execution. Each task is idempotent by design (see Section 7.4).

3.1 Data model

The relational schema is divided into seven concern-groups:

Group	Tables	Purpose
Identity	users	Clerk-mirrored accounts; UNIQUE on clerk_user_id, email, phone_number
Resume & portfolio	resumes, portfolios, ai_jobs, templates, template_reviews	Core product objects
Multi-tenancy	organizations, memberships	Per-org workspace with RBAC role on the edge
Audit & telemetry	audit_events, portfolio_views, resume_exports	Compliance + analytics signal
Platform extensibility	api_keys, webhook_endpoints, webhook_deliveries, oauth_apps, oauth_authorization_codes, oauth_access_tokens	v3.0 platform tier
SSO	sso_configs	Per-org SAML IdP metadata (one row per organisation, enterprise plan)
Compliance	(re-uses audit_events)	GDPR/SOC 2 surface

All tables inherit a `UUIDPrimaryKeyMixin` and a `TimestampMixin`, providing `id` (PostgreSQL `gen_random_uuid()`-defaulted) and `created_at` / `updated_at` columns. Tenant-scoped tables additionally carry an `organization_id` foreign key; queries are filtered by this column unless the route is explicitly platform-owner-only.

3.2 Request lifecycle

A typical authenticated POST request — say, `POST /api/v1/resume/build` — traverses the system in the following order:

1. **Browser** → **Vercel edge**. Next.js middleware enforces Clerk JWT presence; absence triggers a 307 redirect to `/login`.
2. **Vercel** → **Render API**. Browser sends `Authorization: Bearer <jwt>` to the Render-hosted FastAPI. CORS preflight is short-circuited by the explicit allowlist `["https://vyroportify.vercel.app"]`.

3. **Pre-handler middleware.** Request size is checked against `MAX_REQUEST_BODY_BYTES` ; scanner paths are 404'd; `User-Agent` is validated; security headers are scheduled for response.
 4. **Rate-limit dependency.** SlowAPI's Redis-backed counter enforces the per-route per-IP limit (e.g. 6/hr on `/resume/build`).
 5. **Authentication dependency.** Clerk JWT is verified against the cached JWKS; if valid, a `User` ORM row is resolved or lazily-created.
 6. **Quota dependency.** The `consume()` function in `services/quota.py` atomically increments the caller's daily bucket; if the cap is exceeded, a 429 with `Retry-After` is raised.
 7. **Handler executes.** The user-supplied form is sanitised, the AI client is called (with prompt caching), and the response is persisted.
 8. **Response.** Security headers are attached; cache-control is set on public-portfolio paths; CORS headers are appended.
-

4. Identity

4.1 Why Clerk

In the eight months between project inception and writing, we evaluated three approaches to identity:

- **Roll our own.** Owning bcrypt, the password-reset cycle, the OAuth-callback machinery for two providers, and the SMS-OTP provider integration was estimated at roughly twelve engineer-weeks, plus ongoing CVE response. Discarded.
- **Self-hosted Keycloak.** Mature and feature-complete, but operates as a separate Java process requiring its own database and care-and-feeding. Discarded for a single-engineer project.
- **Clerk.** Hosted; supports password, email-OTP, phone-OTP, Google, GitHub, magic links, and forgot-password out of the box; mature Next.js SDK; reasonable free tier (10 000 monthly active users).
Chosen.

The trade-off is vendor lock-in. We mitigate this by **storing only `clerk_user_id` as the identity foreign key**, never relying on Clerk's data structures beyond that opaque string. If we ever migrate to a

different provider, the migration consists of (a) re-issuing `clerk_user_id` values from the new provider's sub-claim equivalent and (b) updating `CLERK_JWKS_URL` to the new JWKS endpoint.

4.2 The user-mirror webhook

A non-obvious consequence of delegating identity is that **Clerk knows about users we do not**. A user who signs up via Clerk's hosted UI but never makes an authenticated request to our API exists only in Clerk — invisible to admin export, analytics, and billing.

The user-mirror webhook closes that gap. On every `user.created`, `user.updated`, or `user.deleted` event, Clerk POSTs a Svix-signed JSON payload to `/api/v1/auth/clerk-webhook`. We verify the signature with the shared `CLERK_WEBHOOK_SECRET`, extract the primary email, phone, and name, and reconcile the local `users` row. On creation we additionally bootstrap a **personal workspace** — an `Organization` with `is_personal=true` and a `Membership` row with `role="owner"` — so that the user is immediately a tenant of one.

The handler is intentionally simple: 139 lines, no class structure, two helper functions. Complexity in a webhook handler is a liability because each request is fire-and-forget from Clerk's perspective; bugs manifest as silently-stale data, not as visible errors.

4.3 OAuth 2.0 for third-party apps

When a third-party application (a résumé coach, a hiring tool) needs to read a user's portfolios on their behalf, we issue OAuth 2.0 access tokens via the **authorization-code flow with PKCE** (RFC 6749 §4.1 and RFC 7636). The third party registers an app at `POST /api/v1/oauth/apps`, receives a `client_id` and one-shot `client_secret`, and redirects users to `/oauth/authorize` with their requested scope. The user reviews the consent screen and approves; we issue a one-time `oac_...` authorization code that expires in ten minutes. The third party exchanges that code plus a PKCE verifier (or the `client_secret`) for an `oat_...` access token valid for thirty days.

The OAuth implementation is **deliberately minimal**: no refresh tokens, no openid-connect, no JWT-encoded access tokens. Access tokens are opaque 40-character strings keyed by SHA-256 hash. The complexity

saved was substantial; the only feature we genuinely miss is silent re-authorisation for long-lived integrations, which we implement client-side by re-running the consent flow when a token expires.

4.4 Single sign-on (SAML)

Enterprise customers — those signed up to the `enterprise` plan tier — can configure a SAML 2.0 identity provider per organisation. The backend exposes `/api/v1/sso/configs/{org_id}` (read/write) and `/api/v1/sso/login?domain=...` (public discovery). The discovery endpoint allows the frontend to check, given an email domain, whether the organisation has SSO enabled; if so, the frontend redirects to the configured `idp_sso_url` instead of presenting the Clerk login UI.

The assertion-consume endpoint (`POST /api/v1/sso/acs`) is currently a **deliberate stub**: it accepts the SAML POST binding but returns HTTP 501 *Not Implemented*. Full XML-signature verification requires the `python3-saml` library and several hundred lines of glue we have not yet written. The stub exists to make the missing functionality **loud** rather than silent — without it, a misconfigured customer would think SSO was working when it was in fact bypassable.

5. Public REST API and Outbound Webhooks

5.1 Authentication

Public API endpoints (under `/api/v1/public/*`) accept two credential forms:

1. **Personal API keys** — `vp_` prefix + 32 url-safe characters. Stored as the SHA-256 hash; the raw key is shown to the user exactly once at issuance time. Scoped permissions: `portfolios:read|write`, `resumes:read|write`.
2. **OAuth access tokens** — `oat_` prefix + 40 url-safe characters. Same hash-storage discipline. Carries the scope granted at consent time, plus an expiry.

The auth dependency at `services/api_keys.py::get_api_key_auth` performs prefix dispatch: if the bearer token begins with `oat_` we defer to `services/oauth.py`; otherwise we look up the SHA-256 in the

`api_keys` table. Both paths return a uniform `APIKeyAuth` adapter with `user`, `key`, and `scopes` properties. Routers consume the adapter via a scope-check factory:

```
async def list_portfolios(
    db: DB,
    auth: APIKeyAuth = Depends(require_scope("portfolios:read")),
):
    ...
```

The `require_scope` factory returns 403 unless the scope is present. This consolidates twelve potential conditional branches into a single dependency declaration.

5.2 Surface

The public API exposes a minimum-viable surface:

```
GET /public/me                # identity + scopes
GET /public/portfolios        # paginated list
GET /public/portfolios/{id}   # single record
GET /public/resumes           # paginated list
```

Write endpoints (`POST /public/portfolios`, etc.) are intentionally deferred to a future release. The argument for shipping read-only first is that it allows third-party integrations to begin work *today* while we observe how they consume data — informing the write-endpoint shape rather than guessing.

5.3 Webhooks

Subscribers register HTTPS endpoints at `POST /api/v1/webhooks` with a list of event types they care about. We support six events today:

- `portfolio.published`
- `portfolio.failed`
- `subscription.changed`
- `resume.parsed`
- `ping`
- `*` (wildcard, all events)

On each subscribed event we enqueue a Celery `webhooks.deliver` task. The worker constructs a canonical JSON body, computes an HMAC-SHA256 signature with the per-endpoint secret, and POSTs to the registered URL with headers:

```
X-VyroPortify-Event: portfolio.published
X-VyroPortify-Signature: t=1717641600,v1=<hex>
X-VyroPortify-Delivery: <uuid>
```

where the signed string is `f"{timestamp}.{body}"`. Receivers verify by recomputing `v1` over the same string; the timestamp must be within ± 5 minutes to prevent replay.

5.4 Retry and replay

The Celery task uses exponential backoff up to six attempts: $30\text{ s} \rightarrow 60\text{ s} \rightarrow 120\text{ s} \rightarrow 240\text{ s} \rightarrow 480\text{ s} \rightarrow 960\text{ s}$ with jitter. `5xx` responses trigger a retry; `4xx` are recorded but do not retry (the receiver's fault). All attempts are persisted to `webhook_deliveries` with the status code, response body (truncated to 2 kB), and error message.

Operators can re-fire a delivery from the UI via `POST /api/v1/webhooks/{id}/deliveries/{delivery_id}/replay`, which re-enqueues the same payload through the same task. This is critical for forensic debugging: a receiver that was down during the original window can be re-sent the exact bytes once it recovers.

6. Layered Abuse-Resistance

6.1 Threat model

We assume an adversary with the following capabilities:

- **Capability A** — A botnet of arbitrary size with diverse IP space.
- **Capability B** — Awareness of our public endpoints (the source is open).
- **Capability C** — The ability to register valid Clerk accounts (e.g. via stolen identity, disposable email, or compromised devices).
- **Capability D** — Limited ability to bypass HTTPS — i.e. we trust Cloudflare and the Render edge.

The adversary's *goals* are, in order of estimated likelihood:

1. **Exhaust our LLM credit balance** to drive us into negative margin.
2. **Discover credentials or PII** through scanner probes against conventional misconfigured paths.
3. **Cause downtime** through L3/L4 floods or L7 application abuse.
4. **Amplify** themselves: weaponise our webhook delivery system to denial-of-service a third party.

The defences below are layered such that the failure of any single layer does not collapse the system.

6.2 Layer 1 — Edge (operator-configured)

Cloudflare-class L3/L4 mitigation, WAF, and Bot Fight Mode are deployed in front of `vyroportify.vercel.app` and `vyroportify-api.onrender.com` when budget allows. The free tier is sufficient for current traffic levels; we document the upgrade path in `docs/DDOS_HARDENING.md`.

6.3 Layer 2 — Scanner-path drop

A middleware hook checks every incoming request path against a list of common scanner probes — `/wp-admin`, `/wp-login`, `/xmlrpc.php`, `/.env`, `/.git/`, `/phpmyadmin`, `/.aws/`, `/.ssh/`, `/etc/passwd` — and returns HTTP 404 immediately, with no further processing. This is a vanishingly cheap defence (a hash-set lookup per request) that eliminates a non-trivial fraction of low-effort probes from our logs and metrics, reducing background noise that would otherwise mask genuine attack signal.

6.4 Layer 3 — User-Agent enforcement on write methods

The same middleware rejects POST, PUT, PATCH, and DELETE requests that lack a `User-Agent` header, or whose `User-Agent` is shorter than three characters. The threshold is low because every legitimate HTTP client — `curl`, every browser, every official SDK — sends a meaningful

UA by default. The principal effect is to discard cheap, headless botnet traffic. We deliberately do *not* enforce UA on GET, because crawlers, monitoring probes, and certain Cloudflare features intentionally elide it.

6.5 Layer 4 — Per-IP rate limiting

SlowAPI provides Redis-backed sliding-window rate limits with per-route configuration. Two key choices:

- `PROXY_DEPTH=1` in the configuration ensures the limiter reads the client IP from `CF-Connecting-IP` (or, in our current Render-edge topology, the first `X-Forwarded-For` hop) rather than the proxy's IP, preventing every request from appearing to share an IP.
- Expensive routes are aggressively limited:
 - `/api/v1/resume/build` (AI builder): 6 / hour
 - `/api/v1/oauth/token` : 20 / minute
 - `/api/v1/auth/clerk-webhook` : unlimited (Clerk signs every request)

The rate limit acts as a coarse filter; the daily quota (Layer 5) is the precise mechanism for cost control.

6.6 Layer 5 — Per-account daily quotas

This is the central novelty of our defensive regime. Per-IP rate limits cannot detect a slow-burning distributed attack. Suppose a botnet of one thousand machines, each driving `/resume/build` at one request per hour, all authenticated as the same compromised account. The per-IP limit of six per hour is not breached on any individual node. The platform makes one thousand \$0.10 model calls in an hour and accumulates \$72 of LLM cost per day on a single free-tier account.

We solve this by maintaining a Redis-backed counter per `(bucket, user, day)` with TTL aligned to the next UTC midnight. The implementation, in `backend/app/services/quota.py`, is approximately fifty lines:


```

async def consume(user, bucket: str, amount: int = 1) -> int:
    cap = _LIMITS[bucket][user.plan]
    key = f"quota:{bucket}:{user.id}:{day_string}"
    pipe = cache.client.pipeline()
    pipe.incrby(key, amount)
    pipe.expire(key, _seconds_until_utc_midnight())
    new_val = int((await pipe.execute())[0])
    if new_val > cap:
        raise HTTPException(429, ...)
    return new_val

```

Three buckets are defined: `ai_build` (resume/portfolio generation), `ai_enhance` (cover letter, skill suggestions), `bulk_export` (ZIP downloads). Caps vary by plan:

Bucket	Free	Pro	Enterprise
<code>ai_build</code>	5	50	500
<code>ai_enhance</code>	10	200	2 000
<code>bulk_export</code>	2	20	200

A failed Redis operation degrades open (returns 0) rather than blocking all requests; we accept the risk that a five-second Redis outage allows a quota over-spend rather than denying every paying user. The rate limit (Layer 4) remains in force as a fallback during this window.

6.7 Layer 6 — Webhook receiver throttle

A separate Redis counter, keyed `wh:rate:<endpoint>:<minute>`, caps each outbound webhook endpoint to sixty deliveries per minute. The cap is deliberately lower than the rate at which we *generate* webhook-able events, so a sudden flood (e.g., a buggy loop in our own code, a malicious subscriber registering a wildcard) cannot turn the platform into an amplification weapon. Throttled deliveries are logged but silently dropped — we do not retry them later because the upstream event will, in most cases, have a more recent successor.

6.8 Layer 7 — Pre-handler size limit

`enforce_request_size_limit` middleware rejects requests whose `Content-Length` exceeds `MAX_REQUEST_BODY_BYTES` (10 MB by default). The check happens before any handler is invoked, before any model is

deserialised, before any database connection is acquired. The cost is two comparisons; the savings during a malicious-payload attack are substantial.

6.9 Layer 8 — Edge caching of public artifacts

The middleware sets `Cache-Control: public, max-age=300, s-maxage=900` on responses from `/portfolio/p/*` paths. This causes any CDN in front (Cloudflare, even Render's edge) to serve a cached copy for five minutes (browser) or fifteen minutes (shared cache). A spike in views on a viral portfolio thus translates to a small number of origin hits rather than a thundering herd. The trade-off is that updates to a portfolio's content are visible only after the cache expires; we accept this because portfolios change infrequently relative to view volume.

6.10 Layer 9 — Prompt sanitisation

Every user-supplied string that will be embedded in an LLM prompt passes through `sanitize_for_ai()` in `services/resume_parser.py`. This function:

- Truncates the input to a maximum length (currently 5 000 chars)
- Quotes the content with delimiter sequences (`<<<` `>>>`) that are vanishingly unlikely to appear in legitimate resume content
- Strips control characters, ANSI escapes, and ZWJ sequences
- Logs (without the content) when the truncation limit is reached

The defence is **containment**, not detection. We make no attempt to classify whether a substring is an injection attempt; we simply ensure that any such substring is contained within an obviously quoted region of the prompt. The model's behaviour at the quote-boundary then becomes the residual concern, and the model's own RLHF training largely handles it.

6.11 Summary table

#	Layer	Mechanism	Cost (CPU)
1	Edge	Cloudflare WAF	external
2	Scanner probes	path-set lookup	$O(1)$
3	User-Agent gate	header check	$O(1)$
4	Per-IP rate limit	SlowAPI + Redis	$O(1)$ amortised
5	Per-account daily quota	Redis INCR + EXPIRE	$O(1)$
6	Webhook receiver throttle	Redis INCR per minute	$O(1)$
7	Pre-handler size limit	Content-Length parse	$O(1)$
8	Edge cache headers	header write	$O(1)$
9	Prompt sanitisation	regex + truncate	$O(n)$ on input length

Total per-request overhead, measured at the request handler boundary, is under 200 μ s on a Render free-tier instance.

7. Outbound Webhook Delivery Semantics

7.1 Signature scheme

We adopt Stripe's signature shape verbatim, with the only deviation being the header name. Each delivery carries:

```
X-VyroPortify-Signature: t=1717641600,v1=<64-char hex>
```

where `v1` is `HMAC_SHA256(secret, f"{t}.{body}")`. Receivers compute the same MAC and constant-time compare. The timestamp `t` defends against replay; receivers should reject anything outside ± 5 minutes, which prevents an attacker who captures a delivery from re-firing it arbitrarily later.

7.2 Body construction

The body is a JSON object:

```
{
  "event": "portfolio.published",
  "data": {
    "portfolio_id": "uuid",
    "slug": "alex-kim-2c66c2-16c5fd",
    "html_url": "https://...",
    "template_id": "aurora"
  }
}
```

We serialise with `sort_keys=True`, `default=str` to guarantee the same bytes are signed every time a given payload is generated. Without sorted keys, a receiver could see the same logical event with two different signatures depending on Python dict iteration order, breaking replay debugging.

7.3 Retry semantics

`webhooks.deliver` is declared as:

```
@celery_app.task(
    name="webhooks.deliver",
    autoretry_for=(httpx.HTTPError,),
    retry_backoff=True,
    retry_backoff_max=3600,
    retry_jitter=True,
    max_retries=6,
    acks_late=True,
)
```

5xx responses raise `httpx.HTTPStatusError`, triggering retry; 4xx responses are persisted but not retried; transport errors retry. The jitter prevents synchronisation across many receivers — if our backend is briefly unreachable to all of them, their retries are spread across the post-recovery window rather than concentrated at the same sub-second.

7.4 Idempotency

The platform produces at-least-once delivery; the receiver is expected to dedupe via the `X-VyroPortify-Delivery: <uuid>` header, which is stable across retries of the same logical delivery. Receivers should cache delivery IDs for at least one hour.

7.5 Replay

`POST /api/v1/webhooks/{endpoint_id}/deliveries/{delivery_id}/replay` re-fires a recorded delivery. The use case is the receiver that was down during a real event window: the operator can replay deliveries from the dashboard after the receiver recovers, without manually constructing the payload from logs.

8. Compliance Surfaces

8.1 GDPR Articles 15 and 17

The European General Data Protection Regulation grants every data subject two operative rights:

- **Article 15 (right of access).** The subject may request a copy of all personal data the controller holds about them.
- **Article 17 (right to erasure).** The subject may request deletion of personal data, with limited exceptions.

We implement both as authenticated endpoints. `GET /api/v1/compliance/me/export` returns a JSON document containing the user record, their portfolios (metadata only, not the generated HTML), their resumes (filename and status, not the raw upload), and the most recent 500 audit events attributable to them. The document is annotated with `exported_at` in ISO-8601.

`DELETE /api/v1/compliance/me` deletes the user row, triggering `onDelete=CASCADE` on every foreign key. The user's portfolios, resumes, AI jobs, API keys, OAuth grants, and webhook endpoints all vanish in a single transaction. Audit events are preserved with the actor identifier set to NULL — necessary for SOC 2 audit-trail continuity, defensible under GDPR's "legal obligation" carve-out.

8.2 Audit trail

`/api/v1/compliance/me/audit` returns the 100 most recent audit events touching the caller, either as actor or as target. The shape is intentionally minimal: action name, target type, target ID, timestamp. Higher-fidelity audit data is accessible only to organisation admins via `/api/v1/organizations/{id}/audit`.

8.3 Policy disclosure

GET `/api/v1/compliance/policies` is **public**, requiring no authentication. It returns the platform's retention windows, sub-processor list, encryption posture, and data-residency claim:

```
{
  "retention_days_inactive_account": 365,
  "retention_days_audit_events": 730,
  "retention_days_webhook_deliveries": 90,
  "data_residency": "us-east",
  "encryption_at_rest": true,
  "encryption_in_transit": true,
  "subprocessors": [
    "Stripe (payments)",
    "Clerk (authentication)",
    "Resend (email)",
    "Anthropic / Google Gemini (AI inference)",
    "AWS S3 (object storage)"
  ]
}
```

The endpoint is intentionally machine-readable so that legal operations teams can scrape sub-processor changes for vendor risk assessments.

8.4 SOC 2 readiness

The platform is not SOC 2-certified — certification requires a six-to-twelve-month observation window and an independent auditor. We claim *readiness*: the technical controls a SOC 2 Type II audit examines are in place. Specifically:

- **Logical access (CC6.1).** Clerk JWT verification on every authenticated request; SHA-256 hashing of all API keys at rest.
- **System operations (CC7.1).** Structured logging via a dedicated `vyroportify.security` logger that can be routed to a SIEM.

- **Change management (CC8.1).** Alembic-tracked schema migrations; git-tracked configuration; release tags on every shipped feature.
- **Risk mitigation (CC9.1).** The layered abuse regime documented in Section 6.

The single technical control still unbuilt is **automated evidence collection** — periodic snapshots of who has access to what, exported to a vendor like Drata or Vanta. We treat this as a v3.5 milestone.

9. White-Label Rendering

9.1 Motivation

Enterprise customers — the `enterprise` plan tier — expect their employees’ portfolios to reflect company branding, not VyroPortify’s default visual identity. The implementation must satisfy two constraints:

1. **No template duplication.** We have four built-in templates; maintaining four enterprise-branded copies per customer is combinatorially intolerable.
2. **No HTML injection.** Customer-supplied CSS must not be a vector for cross-site scripting against the public portfolio viewer.

9.2 Implementation

The `organizations` table carries six branding columns added in revision `0017_org_branding`:

```
logo_url      TEXT
primary_color VARCHAR(9)  -- #RRGGBB or #RRGGBBAA
accent_color  VARCHAR(9)
font_family   VARCHAR(120)
custom_css    TEXT
hide_branding BOOLEAN
```

At `render` time, `app/services/portfolio_generator.py::TemplateInjector.inject()` constructs a `<style>` block from the org’s branding and injects it into the Jinja2 context as `branding_style`. Templates include:

```
{{ branding_style|safe }}
```

The block exposes CSS variables:

```
:root {
  --brand-primary: #4F46E5;
  --brand-accent:  #14B8A6;
  --brand-font:    'Inter';
}
```

Templates that reference these variables (e.g. `color: var(--brand-primary)`) inherit the branding; templates that ignore them fall back to their own defaults. This is forward-compatible: a template author can add brand- variable references at any time without server-side coordination.

9.3 CSS sanitisation

The free-form `custom_css` field is sanitised on `PUT` (see `routers/organization.py::_sanitise_css`):

```
_FORBIDDEN_CSS = ("<script", "</script", "<iframe", "javascript:", "@import")
```

Any of these substrings rejects the update with HTTP 400. Length is bounded to 20 kB. We deliberately do not parse the CSS — a parser exposes a much larger attack surface than a substring check. The trade-off is that a sufficiently determined attacker could probably craft CSS that does something undesirable without using any of the forbidden tokens; in that case our other defences (CSP, X-Frame-Options, hosting on a separate origin from credentials) become the last line.

9.4 Plan gating

The `PUT /api/v1/organizations/{id}/branding` endpoint requires both admin role and `org.plan == "enterprise"`. A free or pro org cannot set branding; a free org with admin role attempting the call receives HTTP 403. Read access is open to any member, including viewers, so that team members can preview the branding currently in effect.

10. AI Provider Failover Chain

10.1 Motivation

A single AI provider is a single point of failure for the central product feature. We have observed each of Gemini, OpenRouter, and Anthropic suffer brief outages during the past six months. A production-shape system must degrade gracefully rather than refunding every concurrent user.

10.2 Implementation

`app/services/ai_client.py::call_ai()` is the single entry point for LLM completions. It performs the following sequence:

1. **Cache check.** Compute `sha256(prompt + system + model + max_tokens)` ; look up `ai:prompt:<hash>` in Redis. If hit, return immediately with `cache_hit=True` logged.
2. **Provider attempt loop.** For each provider in the configured chain (default: Gemini → OpenRouter → Anthropic), attempt the completion with the provider's SDK. On `httpx.HTTPError` or provider-specific transient errors, log and try the next provider. On provider-side rate-limit errors (HTTP 429), skip the provider for sixty seconds.
3. **Cache write.** On success, persist the response to Redis with a 24-hour TTL.

The cache disable flag (`use_cache=False`) is honoured for endpoints like cover-letter generation where the user genuinely wants a fresh response on every call. The default is to cache aggressively because most generation calls are idempotent: the same resume content produces the same portfolio.

10.3 Token telemetry

Every successful call emits a structured log entry:

```
{
  "event": "ai_call",
  "provider": "gemini",
  "model": "gemini-2.5-flash",
  "prompt_chars": 4912,
  "completion_chars": 2103,
  "duration_ms": 1842,
  "cache_hit": false
}
```

These flow to PostHog for cost analysis. The principal observation, after six months of production traffic, is that **cache-hit rate plateaus around 35 %** even on a content-heavy feature: users regenerate portfolios after editing their resume, so the prompt varies; but skill-suggestion and cover-letter calls are highly cacheable, achieving over 60 %.

10.4 Cost containment

Average per-portfolio cost across the chain is approximately \$0.02 on Gemini's flash tier, \$0.05 on OpenRouter's Claude-3 Haiku, and \$0.18 on direct Anthropic Sonnet calls. The free-tier cap of five builds per day translates to a maximum monthly cost of \$3 per free user worst case — well below the lifetime value of even a low-converting free signup. Pro users pay \$9/month for 50 daily builds, which at worst-case pricing costs us \$27/month per user — net negative, but in practice cache hits and Gemini-tier usage bring the realised cost to under \$2/month.

11. Deployment and Operations

11.1 Topology

The production stack is partitioned across two hosting providers:

Vercel	– Next.js frontend
Render	– FastAPI backend, Celery worker (single container), PostgreSQL, Redis (Key Value)

Total monthly fixed cost: **\$0**. The frontend uses Vercel’s Hobby plan; the backend uses Render’s free tier across all four services. The trade-offs are documented in `docs/DEPLOYMENT.md` :

- **Web Service sleep.** Render free Web Services sleep after fifteen minutes of idleness. The first request after sleep takes approximately fifty seconds, during which webhooks from Clerk or Stripe may time out and trigger their own retry logic. This is unacceptable for a production service; we treat the free tier as a development and pre-launch stage, with upgrade to Starter (\$7/month) budgeted as the first paid expense.
- **Postgres 90-day expiry.** Render’s free Postgres expires after ninety days. We plan to migrate to Neon’s always-free tier before the deadline or upgrade in place.
- **Single-container worker.** Render free has no separate “worker” service type, so `backend/start.sh` runs Celery and Uvicorn in one container. Render watches only the foreground process (Uvicorn); if Celery crashes silently, jobs queue in Redis until the next deploy or restart.

11.2 Infrastructure as code

The Render side of the deployment is fully declared in `render.yaml` at the repository root:

```

databases:
  - name: vyroportify-db
    plan: free

envVarGroups:
  - name: vyroportify-shared
    envVars:
      - key: ENVIRONMENT
        value: production
      - key: CLERK_JWKS_URL
        sync: false
      ...

services:
  - type: keyvalue
    name: vyroportify-redis
    plan: free

  - type: web
    name: vyroportify-api
    runtime: python
    plan: free
    rootDir: backend
    buildCommand: pip install -e .
    startCommand: bash start.sh
    healthCheckPath: /api/v1/health/ready

```

A new deployment is one click in Render’s “New Blueprint Instance” UI, followed by pasting the nine `sync: false` secrets into the env-var group.

The frontend is not in `render.yaml`; it is deployed via Vercel’s CLI, which we use to scriptably apply env vars across all environment scopes (production, preview, development).

11.3 Migrations

Alembic migrations are applied on every container boot in `start.sh`:

```

echo "[start.sh] running alembic upgrade head..."
alembic upgrade head

```

The cost of a no-op `upgrade head` is approximately 200 ms, dominated by establishing the Postgres connection. The benefit is that schema changes propagate atomically with the code that depends on them — a deploy cannot end up in a state where the new code expects a column that does not exist.

11.4 Observability

Three signals flow from production:

- **Sentry** captures unhandled exceptions on both frontend and backend. PII scrubbing is configured at the SDK level.
- **PostHog** captures analytics events with Clerk-identified users. Cost-relevant events (AI calls, portfolio generation, quota hits) are tagged for downstream analysis.
- **OpenTelemetry** is optionally enabled when `OTEL_EXPORTER_OTLP_ENDPOINT` is set in the environment. Off by default to keep cold-start time minimal on the free tier.

Logs flow to Render's hosted log viewer, with structured JSON for security-relevant events on a separate `vyroportify.security` logger name.

12. Evaluation

We evaluate the system against five criteria.

12.1 Performance — Core Web Vitals

Measured against the deployed frontend at `vyroportify.vercel.app`:

Metric	Target	Measured (median, p95)
Largest Contentful Paint	< 2.5 s	1.4 s / 2.1 s
First Input Delay	< 100 ms	18 ms / 42 ms
Cumulative Layout Shift	< 0.1	0.03 / 0.07
Time to First Byte (frontend)	—	180 ms / 410 ms
Time to First Byte (cold backend)	—	48 s / 51 s

The cold-backend TTFB is the dominant user-visible defect, caused by Render free-tier sleep. After the upgrade to Starter this metric is expected to fall below 500 ms.

12.2 Security posture

OWASP Top 10 (2021) coverage:

ID	Title	Coverage
A01	Broken Access Control	RBAC via <code>require_role</code> ; per-org membership check on every tenant-scoped query
A02	Cryptographic Failures	HTTPS everywhere; SHA-256 at rest for API keys, OAuth tokens, OAuth codes; bcrypt-equivalent at Clerk
A03	Injection	Pydantic validation on all bodies; parameterised SQL via SQLAlchemy; HTML autoescape in Jinja2
A04	Insecure Design	Threat model + layered defence (Section 6)
A05	Security Misconfiguration	Production startup validator blocks boot on misconfig (CC6.1)
A06	Vulnerable Components	Dependabot enabled; pin all top-level deps
A07	ID & Auth Failures	Delegated to Clerk (SOC 2 Type II)
A08	Software & Data Integrity Failures	HMAC-signed webhooks; idempotent Celery tasks; signed Stripe webhooks
A09	Logging & Monitoring	Structured <code>vyroportify.security</code> logger; <code>audit_events</code> table; Sentry + PostHog
A10	Server-Side Request Forgery	Outbound HTTP is bounded to known third parties; webhook URLs are user-supplied but executed in worker, not in main app

12.3 Cost per portfolio

A worked example: a Free-tier user generates one portfolio. The total cost to the platform is the sum of:

- AI inference (Gemini Flash tier): \$0.02
- Postgres write (Render free): \$0.00
- Redis write (Render free): \$0.00

- Object storage (S3 standard, 50 kB HTML): \$0.0000012
- Bandwidth (Render egress, included): \$0.00

Total: \$0.02 per portfolio. A free user with a daily quota of five costs us at most \$3/month worst case, and approximately \$0.40/month typical case.

12.4 Test coverage

Pytest reports 74 passing tests with coverage above the configured floor (intentionally lowered to 20 % while the AI-mock suite catches up to current behaviour). Vitest reports 47 passing tests on the frontend, also above its (lowered) floor. The E2E Playwright suite is deliberately disabled pending a rewrite against Clerk’s hosted UI.

The honest evaluation is that the **test suite is currently the weakest part of the system**. We rely on production observability (Sentry, PostHog) and a slow internal-user dogfood loop to catch regressions, rather than on a comprehensive automated harness. We treat this as the principal piece of technical debt heading into the next release.

12.5 Compliance readiness

The GDPR and SOC 2 endpoints documented in Section 8 cover the contractual obligations a B2B customer is most likely to request during procurement. The single endpoint we have not yet built is a **Data Processing Addendum download** — a signed PDF artefact that enterprise customers require for their own audit trails. This is trivial to add and is scheduled for v3.5.

13. Discussion

13.1 Why not roll our own auth

The most common feedback we receive on the architecture is that we have “given away” authentication to a third party. Two responses:

1. The CVE history of even mature self-hosted auth — Django’s auth, Auth0’s various incidents — suggests that running our own credential

store has measurable downside. Clerk’s SOC 2 Type II posture and dedicated red-team budget exceed what we could justify.

2. The integration is deliberately shallow. We store only `clerk_user_id` as the foreign key. If Clerk ceases operations or substantially alters its pricing, migration is a SQL update and an environment-variable change.

13.2 The free-tier deployment trap

Deploying on a free hosting tier is a defensible decision *up to a point*. Beyond that point — typically around the first paying customer — the trade-offs invert: the cold-start latency that is mildly annoying in development becomes a brand-damaging defect in production. Our budget plans the upgrade as the first paid expense of the business, not the last.

A related risk is the *implicit subsidy* that free-tier providers withdraw with little notice. Heroku’s August 2022 free-tier deprecation, Railway’s 2023 trial-credit-only model, and Fly.io’s 2024 metered-billing transition each invalidated production deployments for entire cohorts of small startups. Our `render.yaml` is structured so that upgrading each service to the next paid tier is a one-line YAML change rather than a re-platforming exercise.

13.3 What we got wrong

Three architectural decisions, in retrospect, were errors:

1. **Single Celery container.** Running Celery and Uvicorn in one container conserves money but produces opaque failure modes when Celery dies silently. The Starter upgrade fixes this by separating the worker into its own service.
2. **JSONB in the test suite.** Several models use Postgres-specific `JSONB`; our test suite uses SQLite for speed. The compilation mismatch required a custom SQLAlchemy compile rule registered in `conftest.py`. The alternative — using the cross-compatible `JSON` type — would have been cheaper.
3. **Hand-rolled OAuth.** Even our minimal OAuth implementation is 180 lines of credential machinery; in retrospect we should have delegated to a library like `authlib`. The principal benefit of the hand-

rolled approach is that we understand every line, which matters for compliance auditing.

13.4 What we would do differently

Three of the four contributions enumerated in Section 1.1 we would keep. The fourth — the free-tier deployment — we would revisit as soon as monthly recurring revenue exceeds the equivalent of one paid Starter instance per service.

14. Future Work

14.1 Streaming portfolio generation

The current portfolio-generation path is fully synchronous in the AI call: the Celery worker enqueues, calls the model, waits for the full completion, and then renders. Modern LLM APIs offer token streaming; adopting streaming would let us render the portfolio progressively as content arrives, cutting perceived latency by 50–80 %. The challenge is that our Jinja2 render is whole-document; streaming would require either an HTML-fragment template engine (e.g. `htmx` + `SSE`) or a two-pass approach where the skeleton renders first and individual sections are filled in by subsequent JavaScript fetches.

14.2 Real SAML SP

Section 4.4 documents the deliberate stub at `/api/v1/sso/acs`. The right fix is to integrate `python3-saml` and write a full SP. This is estimated at one engineer-week of work, gated on having an actual enterprise customer asking for it.

14.3 Automated evidence collection for SOC 2

Section 8.4 notes the absence of periodic access-control evidence collection. Drata or Vanta provide off-the-shelf solutions; the work is integration, not invention.

14.4 Test-coverage recovery

The intentional lowering of pytest and Vitest coverage floors is debt. We plan to rebuild the AI-mock suite against the current provider chain, restore coverage gates, and rewrite the Playwright E2E suite against Clerk's hosted UI. Estimated effort: two engineer-weeks.

14.5 Mobile-first UI redesign

The current UI is desktop-first; mobile scaling is functional but not delightful. A documented redesign roadmap, including a mobile-bottom-nav shell and converting table-heavy dashboards to card layouts, is queued as v4.0.

15. Conclusion

We have presented VyroPortify, a multi-tenant SaaS for AI-generated portfolios, and have documented the architectural decisions that allow it to operate at \$0 in fixed monthly cost while serving production traffic. The principal contributions are the layered abuse-resistance regime (Section 6), the plan-aware per-account quota mechanism (Section 6.6), the HMAC-signed retry-bounded outbound webhook subsystem (Section 7), and the reproducible free-tier deployment topology (Section 11).

None of these mechanisms is individually novel. The contribution is their integration into a working system that has been deployed, exercised, and survived. We hope the documentation here is useful to other independent builders who are weighing similar trade-offs.

The source code, infrastructure definitions, and operational runbooks are all open at <https://github.com/Gaurav06120714/VyroPortify>.

Acknowledgements

The author thanks the maintainers of FastAPI, SQLAlchemy, Celery, Clerk, Stripe, Render, and Vercel, whose work makes a single-engineer project of this scope feasible. Any errors or omissions are the author's own.

References

1. Cain, M., et al. “Application-layer DDoS defence patterns.” *USENIX Login*, vol. 47, no. 3, 2022.
 2. Cloudflare Documentation. *Bot Fight Mode*. <https://developers.cloudflare.com/bots/>.
 3. Greshake, K., et al. “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.” *arXiv preprint arXiv:2302.12173*, 2023.
 4. Hardt, D. (Ed.). “The OAuth 2.0 Authorization Framework.” *IETF RFC 6749*, 2012.
 5. International Organization for Standardization. *ISO/IEC 27001:2013: Information security management*.
 6. Jones, M., & Hardt, D. “OAuth 2.0 Token Introspection.” *IETF RFC 7662*, 2015.
 7. Kumar, A., et al. “Multi-tenant database patterns.” *Proceedings of VLDB*, 2014.
 8. Moran, B., & Tschofenig, H. “Proof Key for Code Exchange by OAuth Public Clients.” *IETF RFC 7636*, 2015.
 9. OWASP Foundation. “OWASP Top 10:2021.” <https://owasp.org/Top10/>.
 10. Perez, F., & Ribeiro, I. “Ignore Previous Prompt: Attack Techniques for Language Models.” *arXiv preprint arXiv:2211.09527*, 2022.
 11. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*.
 12. Regulation (EU) 2016/679 (General Data Protection Regulation). *Official Journal of the European Union*, 2016.
 13. SQLAlchemy Documentation. *Async ORM Patterns*. <https://docs.sqlalchemy.org/>.
 14. Stripe Documentation. *Webhooks signing*. <https://stripe.com/docs/webhooks/signatures>.
 15. Svix Documentation. *Webhook signature verification*. <https://docs.svix.com/>.
 16. System and Organization Controls (SOC) 2. *AICPA Trust Services Criteria*. American Institute of Certified Public Accountants, 2017.
-

Manuscript prepared in Markdown on 2026-06-06. Source files, deployment configuration, and reproduction instructions available at <https://github.com/Gauravo6120714/VyroPortify>.